PROVISIONAL PATENT APPLICATION

Title: EIDOLON: Three-Tiered Hub-and-Spoke Dragoncache Architecture for Pre-Broadcast Inter-Process Communication with Integrity-Gated Lane Promotion

---

COVER SHEET

Application Type: Provisional Patent Application (35 U.S.C. § 111(b))

Title of Invention: Three-Tiered Hub-and-Spoke Memory-Mapped Inter-Process Communication Engine with Component Foresight Pre-Broadcast and CRC64-Gated Lane Promotion

Inventor: Scott Slater
700 E. 8th St.
Unit 14E
Kansas City, MO 64106-5400
United States Citizen

Filing Date: May 6, 2026

Docket / Reference: SES-002-EIDOLON

Correspondence Address:
700 E. 8th St.
Unit 14E
Kansas City, MO 64106-5400

FIELD OF THE INVENTION

This invention relates to inter-process communication (IPC) architectures implemented in reconfigurable hardware (FPGA) and digital logic, and more particularly to a three-tiered, hub-and-spoke, memory-mapped IPC engine that pre-broadcasts integrity-sealed data to all connected processing nodes before final delivery, thereby eliminating accumulation latency at the point of consumption.

BACKGROUND

1.1 The Latency Accumulation Problem

In conventional IPC architectures — whether message queues, shared memory ring buffers, or PCIe DMA engines — data delivery is a pull operation: a consumer requests data only when it needs it, and the pipeline stalls while the request propagates upstream, data is retrieved from backing store, integrity is checked, and the response is returned. Each of these steps contributes serialized latency. In a high-frequency real-time system (megahertz-class event rates, nanosecond timing budgets), this accumulated latency is fatal to determinism.

Known techniques address individual symptoms:

- **Double buffering** reduces consumer stall when switching active data sets, but does not eliminate the upstream retrieval latency on the inactive buffer.
- **Pre-fetching** speculatively loads likely-needed data but requires accurate prediction; mispredictions waste bandwidth and can corrupt ordering guarantees.
- **DMA scatter-gather** reduces CPU load for data movement but does not provide pre-delivery to multiple consumers simultaneously.
- **Non-Transparent PCIe bridging** allows point-to-point peer communication across PCIe segments but does not natively provide broadcast semantics or integrity gating.
- **Zero-copy shared memory** eliminates copy overhead but does not pre-process or pre-validate data before the consumer accesses it.

None of the above architectures combine (a) pre-broadcast of fully processed and integrity-sealed data to all connected nodes simultaneously, (b) autonomous lane selection driven by real-time telemetry, (c) a deterministic address-space management engine that prevents fragmentation before it occurs, and (d) a runtime-configurable read discipline (LIFO/FIFO) without RTL recompilation.

1.2 Prior Art Limitations

Conventional ring buffer IPC (e.g., DPDK rte_ring, Linux io_uring, RDMA send queues) operates in a single-producer / single-consumer or multi-producer / multi-consumer model. Broadcast to N consumers requires N separate enqueue operations or N separate read pointers into a shared ring, with no integrity check at the point of lane promotion, and no mechanism for the system to autonomously switch lanes when a fault is detected mid-transmission.

Hardware message-passing architectures (e.g., Intel QuickPath Interconnect, AMD Infinity Fabric) provide high-bandwidth coherent fabric but are tightly coupled to processor microarchitecture and are not reconfigurable for application-specific integrity or lane-selection policies.

Field-programmable gate array (FPGA) IP cores for AXI4-Stream switches and PCIe endpoint DMA engines provide standard-compliant data movement but do not incorporate application-layer integrity gating (e.g., CRC64 seal validation) as a prerequisite for switching from promotion to delivery, nor do they provide component foresight semantics.

1.3 Summary of the Problem

What is needed is a reconfigurable hardware IPC engine that:

1. Structures data flow across three clearly separated tiers with defined entry and exit contracts.
2. Pre-broadcasts fully processed, integrity-sealed data to all connected downstream nodes *before* final delivery, so that each node has already received and verified the data when it begins processing.
3. Gates lane promotion on a cryptographic integrity seal (CRC64), not on a simple status bit.
4. Autonomously selects and switches PCIe delivery lanes in response to real-time telemetry without host CPU intervention.
5. Prevents memory fragmentation via a BRAM-backed address-space management engine integrated directly into the IPC pipeline.
6. Allows runtime reconfiguration of read discipline (LIFO vs. FIFO) without modifying or re-synthesizing RTL.

SUMMARY OF THE INVENTION

Eidolon is a three-tiered, hub-and-spoke, memory-mapped IPC engine implemented in reconfigurable hardware (FPGA) and associated digital logic. The three tiers are designated

Inception (Tier 1), Promotion (Tier 2), and Delivery (Tier 3), each with defined input/output contracts and autonomous internal logic.

The central innovation is component foresight: Eidolon broadcasts fully processed and integrity-sealed data to all connected PCIe spoke endpoints *before* the Conductor FSM authorizes final delivery. Each connected node has already received and internally verified the incoming data before it is "officially" delivered, eliminating the round-trip latency that would otherwise occur at the delivery boundary.

A second critical innovation is CRC64-gated lane promotion: the Conductor FSM will not advance a data unit from Promotion (Tier 2) to Delivery (Tier 3) unless and until the `crc64_seal` module has asserted `seal_intact` for that data unit. This is not advisory — it is a hard gate in RTL logic. A single-bit error anywhere in the data unit will stall promotion until the error is corrected or the unit is discarded and reprocessed.

A third innovation is autonomous lane switching: the Conductor FSM monitors real-time hardware telemetry from the system monitor (XADC/SYSMON) and independently switches PCIe delivery lanes when thermal, voltage, or clock-failure anomalies are detected. No host CPU interrupt or software handler is required on the critical path.

A fourth innovation is the chamber-alternation model: the hub ring buffer operates as two alternating chambers (A and B), controlled exclusively by the Conductor FSM. While Chamber A is being read for Promotion, Chamber B is being written by Inception. The switch point is a deterministic FSM state, not an interrupt, ensuring zero pipeline bubbles at chamber boundaries.

A fifth innovation is the integration of a balanced binary search tree (the extent_tree) implemented in BRAM, operating as a real-time address-space management engine inside the IPC pipeline. The extent_tree tracks contiguous free and occupied address ranges within the hub ring buffer and is queried on every allocation, preventing fragmentation before it can accumulate. This is not a software allocator running on a host CPU — it is deterministic RTL logic operating in the same clock domain as the data path.

A sixth innovation is the lua_register_bridge: a full AXI4-Lite slave register bank that exposes Tier 2 parameters — including read discipline (LIFO vs. FIFO), drain depth, and alarm thresholds — to a host-embedded Lua runtime via PCIe MMIO. The read discipline of the stack_reader (LIFO or FIFO) is changed at runtime by writing to the AXI register, without any RTL modification or re-synthesis.

When combined with the Shimmers oscillator engine (SES-001) and the SPECTRE frequency-domain transceiver (SES-003), Eidolon forms the middle tier of the Shimmers · Eidolon · SPECTRE (SES) integrated system, providing the validated, integrity-sealed pulse stream that SPECTRE ingests.

BRIEF DESCRIPTION OF DRAWINGS

(Informal drawings — formal drawings to accompany non-provisional application)

Figure 1 — Three-Tier Pipeline Overview Shows Inception (Tier 1) on the left feeding data through the hub ring buffer into Promotion (Tier 2), then into Delivery (Tier 3) on the right. PCIe spokes radiate from the Promotion hub to all connected endpoints. The CRC64 seal gate is shown between Promotion and Delivery.

Figure 2 — Hub Ring Buffer with Chamber A/B Alternation Shows the dual-chamber ring buffer with write pointer (Chamber B active write), read pointer (Chamber A active read), and the Conductor FSM chamber_switch signal that controls alternation. Hub occupancy counter feeds both stack_reader and extent_tree.

Figure 3 — Conductor FSM State Diagram States: ST_RESET → ST_INIT → ST_INCEPTION_ACTIVE → ST_SEAL_WAIT → ST_PROMOTE → ST_BROADCAST → ST_DELIVER → ST_CHAMBER_SWITCH → ST_INCEPTION_ACTIVE (loop). Transitions include XADC fault injection (ST_LANE_SWITCH branch) and CRC64 fail path (ST_SEAL_FAIL → ST_INCEPTION_ACTIVE retry).

Figure 4 — Component Foresight Broadcast Timing Waveform diagram showing hub_occupancy rising as Chamber A fills, broadcast_valid asserting to all N PCIe spoke endpoints simultaneously, each endpoint asserting broadcast_ack, then promote_enable asserting only after all ACKs received AND seal_intact is high.

Figure 5 — Extent Tree FSM and BRAM Structure 32-entry flat table (in BRAM): each entry has {valid, start_addr[13:0], end_addr[13:0]}. FSM states: ST_INIT (primes one free extent covering full chamber), ST_IDLE, ST_SCAN_GET, ST_SCAN_ADD, ST_GRANT, ST_FAIL. Linear scan with early-exit on first fitting extent.

Figure 6 — AXI4-Lite Register Map (lua_register_bridge) Four registers at offsets 0x00 (CONTROL: stack_mode, enable), 0x04 (XADC_THRESHOLDS: temp_warn[7:0], volt_low[7:0]), 0x08 (DRAIN_DEPTH: drain_depth[13:0]), 0x0C (STATUS: read-only: fifo_occupancy[13:0], seal_ok, xadc_alert). AWVALID/AWREADY, WVALID/WREADY, BVALID/BREADY handshake shown.

Figure 7 — Stack Reader LIFO vs. FIFO Mode Two timing diagrams: (a) LIFO mode — read_address decrements from fill_depth toward 0; (b) FIFO mode — read_address increments from 0 toward fill_depth. Mode selected by AXI register write at runtime. RTL is identical for both modes — only mode_select signal value differs.

Figure 8 — CRC64 Integrity Gate Detailed View Shows crc64_seal receiving 512-bit inception_data, computing CRC64-ECMA-182 over N cycles, asserting seal_intact when computed CRC matches expected. Conductor FSM held in ST_SEAL_WAIT state until seal_intact rises. Only after seal_intact does the FSM advance to ST_PROMOTE.

Figure 9 — SES System Context (Eidolon as Middle Tier) Shows Shimmers oscillator output (frequency-encoded pulse stream) entering Tier 1 (Inception) of Eidolon, flowing through Promotion and Delivery, and exiting via spectre_handoff to SPECTRE's wav_encoder. Eidolon is the validated IPC backbone that connects the oscillator to the transceiver.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENT

4.1 Architectural Overview — The Three-Tier Dragoncache

Eidolon is implemented as synthesizable VHDL targeting UltraScale+ FPGA fabric (Xilinx xcu250-figd2104-2L-e in the primary validation platform). The architecture is partitioned into three tiers, each implemented as distinct RTL modules with defined port contracts, operating in a shared 100 MHz clock domain except where noted.

The three tiers form a unidirectional pipeline: data enters at Tier 1 (Inception), is promoted and broadcast at Tier 2 (Promotion), and is delivered at Tier 3 (Delivery). Data never flows backward. The Conductor FSM is the sole arbiter of advancement between tiers.

4.2 Novel Contribution 1 — Component Foresight Pre-Broadcast

The central technical innovation of Eidolon is component foresight: the system broadcasts fully processed and integrity-verified data to all connected downstream nodes before the Conductor FSM authorizes final delivery.

In the preferred embodiment, the hub ring buffer (triton_dragoncache_hub) presents completed data units on its read bus. The Conductor FSM asserts `broadcast_valid` to all N PCIe spoke interfaces simultaneously. Each spoke interface receives the data and asserts `broadcast_ack` when its internal staging buffer has captured the unit. Only after all N `broadcast_ack` signals have been asserted AND the CRC64 seal has been verified (`seal_intact = '1'`) does the Conductor FSM advance to ST_DELIVER and assert `promote_enable`.

This means that by the time `promote_enable` asserts, every connected downstream node already holds a verified copy of the data in its local staging buffer. The "delivery" event at Tier 3

is effectively a release signal — not a data transfer. The actual data transfer has already completed, in parallel, during the broadcast phase.

The latency savings are substantial: in a conventional IPC system with N downstream nodes, delivery to the slowest node determines total delivery time. With component foresight, all N transfers happen in parallel before the delivery gate opens, and the delivery event is O(1) regardless of N.

The number of PCIe spokes N is a generic parameter in the hub RTL, ranging from 1 to 16 in the preferred embodiment, with 4 spokes used in the primary validation configuration.

4.3 Novel Contribution 2 — CRC64-ECMA-182 Integrity Seal as Hard Lane Promotion Gate

The Conductor FSM implements a hard integrity gate between Promotion (Tier 2) and Delivery (Tier 3). The gate is not advisory — it is a combinational interlock in RTL.

The seal computation: The `crc64_seal` module receives the 512-bit inception data word as its input. It computes the CRC64-ECMA-182 polynomial (0x42F0E1EBA9EA3693) over the entire data unit, producing a 64-bit check value. If the received data unit was assembled without corruption, the computed check value matches the expected seal value transmitted alongside the data. If it does not match, `seal_intact` remains low indefinitely for that data unit.

Why CRC64, not CRC32: CRC32 has a collision probability of approximately 1 in 4 billion for random 512-bit data. In a system generating millions of data units per second over a multi-hour benchmark run, this produces a statistically likely undetected error. CRC64-ECMA-182 reduces the collision probability to approximately 1 in 18 quintillion — effectively zero for any practically realizable workload.

The gate implementation: In the Conductor FSM, the transition from ST_SEAL_WAIT to ST_PROMOTE has the condition:

next_state <= ST_PROMOTE when (seal_intact = '1' and all_ack_received = '1') else ST_SEAL_WAIT;

There is no timeout, no bypass, and no software override for this gate. A data unit that fails CRC64 validation is either retransmitted from Inception or discarded, and the Conductor FSM returns to ST_INCEPTION_ACTIVE. The integrity of every delivered data unit is guaranteed by construction.

4.4 Novel Contribution 3 — Conductor FSM with Autonomous Lane Switching

The Conductor FSM is the central control plane of Eidolon. It operates entirely within FPGA fabric with no host CPU on the critical path. Its primary functions are: (a) chamber alternation control, (b) CRC64 gate enforcement, (c) broadcast sequencing, (d) deliver authorization, and (e) autonomous lane switching.

Autonomous lane switching is triggered by the `xadc_alert` signal from the system monitor (XADC/SYSMON) module. When thermal or voltage anomalies exceed configured thresholds, the system monitor asserts `xadc_alert`. The Conductor FSM detects this assertion in any state and transitions to ST_LANE_SWITCH. In ST_LANE_SWITCH, the FSM:

1. Completes the current chamber's in-progress operation if it is in an interruptible phase.
2. Asserts `lane_switch_req` to the PCIe endpoint arbiter, requesting assignment of the next available healthy lane.
3. Waits for `lane_switch_ack` from the arbiter.
4. Updates its internal lane index register.
5. Returns to ST_PROMOTE (if the data unit was sealed and broadcast) or ST_INCEPTION_ACTIVE (if the anomaly arrived before sealing).

This behavior ensures that a hardware fault on one PCIe lane does not stall or crash the IPC engine — the system migrates autonomously to a healthy lane within a deterministic number of clock cycles. No interrupt service routine, no device driver, no operating system involvement.

A clock-failure event (`clock_fail_in`) triggers a separate but similar path: the FSM asserts `fscm_switch_req` and waits for the fabric clock switching module to present an alternate clock source before resuming. This is a clock-failure survival mechanism, not a shutdown handler.

4.5 Novel Contribution 4 — Chamber A/B Alternation Under Exclusive FSM Control

The hub ring buffer operates as two alternating chambers. Chamber A and Chamber B are not physical duplicates — they are logical divisions of the same BRAM array, addressable via distinct base address offsets. The Conductor FSM maintains a `active_chamber` register (1-bit: '0' for A, '1' for B) and asserts `chamber_switch` when it transitions from ST_DELIVER back to ST_INCEPTION_ACTIVE.

The invariant: At all times, exactly one chamber is the write target (being filled by Tier 1 Inception) and the other is the read source (being consumed by Tier 2 Promotion). This invariant is maintained exclusively by the Conductor FSM — no external signal, no software write, and no

other RTL module can alter `active_chamber`. The BRAM write enable and read address are multiplexed based on `active_chamber`.

The switch event: At the ST_DELIVER → ST_INCEPTION_ACTIVE transition, the Conductor FSM inverts `active_chamber` and asserts `chamber_switch` for one clock cycle. This notifies Tier 1 that a new chamber is now available for writing and simultaneously redirects the Tier 2 read path to the newly completed chamber. There are zero pipeline bubbles at the switch point.

The hub occupancy counter tracks the number of valid data units in the currently-promoted chamber. It resets to zero on `chamber_switch` and increments with each valid Inception write. The stack_reader and extent_tree both receive this counter as their primary sizing input.

4.6 Novel Contribution 5 — BRAM-Backed Balanced Extent Tree for In-Pipeline Address Space Management

The `extent_tree` module is a real-time address space management engine implemented entirely in FPGA BRAM, operating in the same 100 MHz clock domain as the data path. It tracks contiguous free and occupied address ranges within the hub ring buffer chamber, preventing memory fragmentation before it occurs.

Implementation: The extent_tree uses a 32-entry flat table stored in synchronous BRAM (not in logic registers, to avoid consuming LUT resources for what is fundamentally a memory operation). Each table entry contains three fields: `valid` (1 bit), `start_address` (14 bits), and `end_address` (14 bits). The 14-bit address width accommodates ring buffers up to 16,384 entries deep.

Initialization: When the Conductor FSM asserts `chamber_switch`, the extent_tree FSM enters ST_INIT. In ST_INIT, it writes a single entry to the table: `{valid=1, start=0, end=chamber_depth-1}`, representing the entire new chamber as one contiguous free extent. This primes the allocator in one clock cycle.

Allocation (get operation): When Tier 1 requests an address range of length L (via `get_valid` assertion and `get_length` input), the extent_tree FSM scans the table linearly for the first entry with `end - start + 1 >= L`. On finding a match, it: (a) asserts `grant` and presents `start_out` and `end_out` to the requester, (b) splits the matched entry into an allocated portion and a remaining free portion (if any), and (c) updates the table in BRAM. If no matching extent is found, it asserts `get_fail`.

Deallocation (add operation): When a chamber is retired via chamber_switch, the extent_tree does not need to perform deallocation — the entire chamber is re-initialized to one free extent in ST_INIT. This eliminates the fragmentation accumulation problem: the allocator always starts each chamber from a clean slate.

Why this is novel: Conventional FPGA IPC engines either use fixed-offset addressing (no allocator, no fragmentation protection) or delegate allocation to a software host (adding interrupt latency on every allocation). The extent_tree is a hardware allocator operating deterministically in the IPC critical path with a bounded worst-case latency of 32 BRAM read cycles (one scan of all 32 entries).

4.7 Novel Contribution 6 — Stack Reader with Runtime-Configurable LIFO/FIFO Discipline

The `stack_reader` module implements a configurable read engine for the hub ring buffer. Its read discipline — LIFO (last-in, first-out) or FIFO (first-in, first-out) — is controlled by a single 1-bit `mode_select` signal that can be changed at runtime without RTL modification or re-synthesis.

LIFO mode: `read_address` is initialized to `fill_depth - 1` and decrements on each `read_enable` assertion. This produces reverse-order readout from the most recently written entry to the oldest. LIFO ordering is optimal for temporal locality: the most recently written data is most likely to be most relevant to the current processing context.

FIFO mode: `read_address` is initialized to 0 and increments on each `read_enable` assertion. This produces in-order readout from oldest to newest entry, matching the natural pipeline order of arrival.

The mode_select signal is driven directly by the `stack_mode_out` output of the `lua_register_bridge` AXI register. The Lua host runtime writes to the AXI register; the register output immediately drives `mode_select`. The stack_reader sees the new mode on the next clock edge following the AXI write. No pipeline flush is required — the `read_reset` input (driven by `chamber_switch`) resets the read pointer to the appropriate starting position on the next chamber's data.

`read_empty` asserts when `read_address` has exhausted the fill range. `read_valid` asserts one cycle after each `read_enable` while data is available.

4.8 Novel Contribution 7 — AXI4-Lite Register Bridge Exposing Tier 2 Parameters to Lua Host Runtime

The `lua_register_bridge` is a full AXI4-Lite slave peripheral implemented in synthesizable VHDL. It provides host software (specifically the embedded Lua 5.4 runtime running in the Trident C++ host driver) with read/write access to Tier 2 operational parameters via PCIe MMIO.

Register map:

- **0x00 CONTROL:** `stack_mode` (bit 0, read/write) — selects LIFO (1) or FIFO (0) for stack_reader. `tier2_enable` (bit 1, read/write) — gates Promotion-tier processing.
- **0x04 XADC_THRESHOLDS:** `temp_warn_threshold[7:0]` (bits 7:0, read/write) — temperature alarm threshold forwarded to Conductor FSM. `volt_low_threshold[7:0]` (bits 15:8, read/write) — voltage floor threshold.
- **0x08 DRAIN_DEPTH:** `drain_depth[13:0]` (bits 13:0, read/write) — maximum number of entries the stack_reader will drain before asserting `read_empty`, used to control burst sizing.
- **0x0C STATUS:** `fifo_occupancy[13:0]` (bits 13:0, read-only) — current hub ring buffer occupancy. `seal_ok` (bit 14, read-only) — current value of `seal_intact`. `xadc_alert_status` (bit 15, read-only) — current value of `xadc_alert`.

AXI handshake: The bridge implements the standard AXI4-Lite write path (AWVALID/AWREADY, WVALID/WREADY, BVALID/BREADY) and read path (ARVALID/ARREADY, RVALID/RREADY) with separate address and data channels. Write acceptance occurs in a two-state FSM: ST_IDLE → ST_WRITE_ACCEPT. Read acceptance occurs similarly. All register outputs are synchronous on the 100 MHz fabric clock.

Integration with Lua runtime: The Lua host runtime calls into the C++ host driver via the existing `src/interpreter/interpreter.cc` interface. Custom Lua functions (e.g., `set_stack_mode("lifo")`) map directly to MMIO writes to the `lua_register_bridge` via the PCIe BAR mapping. This allows operational reconfiguration of Eidolon's Tier 2 behavior from a Lua script without modifying, recompiling, or re-synthesizing RTL.

4.9 Novel Contribution 8 — Three-Tier Pipeline Separation with Hard Interface Contracts

The Eidolon architecture enforces strict tier separation through hard interface contracts in RTL. Data flows in one direction only. No module in Tier N may drive a signal that feeds back into Tier N-1. The Conductor FSM is the only entity permitted to assert control signals that cross tier boundaries.

Tier 1 → Hub interface contract:

- `inception_data[511:0]`: 512-bit data word, valid when `inception_valid = '1'`.
- Hub asserts `hub_write_ack` when data is accepted. Tier 1 must hold data stable until ack.
- Tier 1 is not permitted to observe any Hub internal signals except `hub_occupancy` (for flow control).

Hub → Tier 2 interface contract:

- Tier 2 reads via `read_address[13:0]` presented to hub BRAM port. Hub presents `hub_read_data[511:0]` on the following clock cycle.
- Tier 2 receives `hub_occupancy[11:0]` for stack_reader and extent_tree sizing.
- Tier 2 does not observe Tier 1 signals.

Tier 2 → Tier 3 interface contract:

- Conductor FSM asserts `promote_enable` only when `seal_intact = '1'` and all `broadcast_ack` signals are high.
- Tier 3 (`uart_intake`, `pulse_injector`, `spectre_handoff`) asserts `delivery_ready` when prepared to receive.
- `promote_enable` and `delivery_ready` are the only signals crossing the Tier 2 → Tier 3 boundary in the data path.

These contracts are enforced structurally in the top-level wiring of `top_triton_system.vhdl` — signals are simply not connected across tier boundaries except through the defined contract ports.

4.10 Integration as Part of the SES Compiled Architecture

When Eidolon operates as part of the Shimmers · Eidolon · SPECTRE (SES) integrated system:

- **Shimmers feeds Eidolon Tier 1:** The oscillator engine's pulse output stream is the data source for Inception. Each 512-bit inception data word is constructed from the OSERDESE3 output stream, byte-reversed by the endian_converter, and Huffman-encoded by huff_encoder before entering the hub ring buffer.

- **Eidolon feeds SPECTRE:** The Delivery tier's `spectre_handoff` interface presents the validated, integrity-sealed pulse stream in the exact format expected by SPECTRE's `wav_encoder` and `frequency_analyzer` modules: 32-bit IEEE 754 float samples at 96 kHz, mono. No format conversion is required at the handoff boundary.

- **Emergent system property:** When the three components are integrated, the SES system exhibits component foresight at the SPECTRE level: SPECTRE's Goertzel analyzer is already staging its frequency-detection window by the time Eidolon asserts `promote_enable`. The 5 ms Goertzel window (480 samples at 96 kHz) is fully pre-loaded before the SPECTRE decode stage begins, eliminating the window-fill latency from the critical path.

CLAIMS

(Informal claims for provisional filing — formal claims to be developed with patent attorney for non-provisional application)

Claim 1. A hardware inter-process communication engine comprising: a first processing tier (Inception) that ingests raw data and writes integrity-annotated data units to a hub ring buffer; a second processing tier (Promotion) that reads data units from the hub ring buffer, computes an integrity seal over each data unit, and broadcasts the data units to a plurality of downstream processing nodes via a hub-and-spoke interconnect before delivery is authorized; and a third processing tier (Delivery) that receives a delivery authorization signal and releases previously broadcast data units from downstream staging buffers to active processing pipelines.

Claim 2. The engine of Claim 1, wherein the delivery authorization signal is conditioned on all of: (a) assertion of a cryptographic integrity seal valid signal computed by a CRC64-ECMA-182 module, and (b) assertion of a broadcast acknowledgment signal by every downstream processing node in the plurality.

Claim 3. The engine of Claim 1, wherein a Conductor finite state machine (FSM) implemented in reconfigurable hardware (FPGA) is the sole arbiter of state transitions between tiers, and wherein no host CPU intervention is required on the critical data path.

Claim 4. The engine of Claim 3, wherein the Conductor FSM autonomously switches the active PCIe delivery lane from a faulted lane to a healthy lane in response to a hardware telemetry alert signal from a system monitor module, without host CPU interrupt or software handler involvement.

Claim 5. The engine of Claim 1, wherein the hub ring buffer is partitioned into two alternating logical chambers (Chamber A and Chamber B), and wherein the Conductor FSM exclusively controls chamber alternation by asserting a chamber-switch signal, such that exactly one chamber is being written by the Inception tier and the other is being read by the Promotion tier at all times.

Claim 6. The engine of Claim 5, further comprising a balanced binary search tree (extent_tree) implemented in BRAM, operating in the same clock domain as the hub ring buffer, that tracks contiguous free and occupied address ranges within the active ring buffer chamber and prevents memory fragmentation by re-initializing to a single contiguous free extent on each chamber-switch event.

Claim 7. The engine of Claim 1, further comprising a stack reader module that produces read addresses for the hub ring buffer in either last-in, first-out (LIFO) or first-in, first-out (FIFO) order,

wherein the read discipline is selected by a mode-select signal that is driven by an AXI4-Lite register and changeable at runtime without RTL modification or re-synthesis.

Claim 8. The engine of Claim 7, further comprising an AXI4-Lite register bridge that exposes operational parameters of the Promotion tier to a host-embedded Lua runtime via PCIe memory-mapped I/O (MMIO), including at minimum: read discipline mode, drain depth, thermal and voltage alarm thresholds, and read-only ring buffer occupancy and integrity seal status.

Claim 9. The engine of Claim 1, wherein the downstream processing nodes receive fully integrity-verified copies of each data unit during the broadcast phase of the Promotion tier, such that the Delivery tier authorization event requires no data transfer and constitutes only a release of data already staged in each node's local buffer.

Claim 10. The engine of Claim 1, wherein the integrity seal is computed using the CRC64-ECMA-182 polynomial (0x42F0E1EBA9EA3693) over a 512-bit data word, and wherein a CRC64 integrity check failure causes the Conductor FSM to return to the Inception tier state and prevents delivery without exception, bypass, or software override.

Claim 11. A method of pre-broadcasting inter-process communication data in reconfigurable hardware, comprising: receiving raw data into a first hardware processing tier; computing a 64-bit cyclic redundancy check over each data unit using the ECMA-182 polynomial; simultaneously transmitting the data unit to a plurality of downstream hardware nodes via a hub-and-spoke interconnect before authorizing final delivery; waiting for acknowledgment from all downstream nodes and validity of the integrity check; and asserting a delivery authorization that releases the pre-staged data in each downstream node's local buffer without requiring a further data transfer.

Claim 12. An integrated system comprising the engine of Claim 1, an oscillator engine that generates frequency-encoded digital pulse streams and supplies them to the Inception tier of the engine, and a frequency-domain data transceiver that receives validated pulse streams from the Delivery tier of the engine and decodes them via Fourier-transform-based frequency analysis, wherein the oscillator engine, the engine, and the transceiver are implemented in a common reconfigurable hardware fabric and operate without host CPU intervention on the data path.

ABSTRACT

A three-tiered, hub-and-spoke, memory-mapped inter-process communication (IPC) engine implemented in reconfigurable hardware (FPGA) that introduces component foresight as its central architectural innovation: data is broadcast to all connected downstream nodes and integrity-verified before delivery is authorized, so that every consumer has already staged and

verified incoming data by the time the delivery gate opens. The engine partitions processing into three tiers — Inception, Promotion, and Delivery — controlled by a Conductor finite state machine that gates tier advancement on a CRC64-ECMA-182 integrity seal. A BRAM-backed extent tree prevents ring buffer fragmentation in the data path. A stack reader provides runtime-switchable LIFO or FIFO read discipline without RTL recompilation. An AXI4-Lite register bridge exposes Tier 2 operational parameters to an embedded Lua runtime via PCIe MMIO. The Conductor FSM autonomously switches PCIe delivery lanes on hardware fault detection without host CPU intervention. When combined with a frequency oscillator engine and a frequency-domain transceiver, the engine forms the validated IPC backbone of an integrated data processing system.

INVENTOR DECLARATION

I hereby declare that I am the sole inventor of the subject matter claimed in this provisional patent application; that I believe I am the original and first inventor of the invention; that I have reviewed and understand the contents of this application; and that I acknowledge my duty to disclose information material to patentability.

All artificial intelligence tools used during the development and documentation of this invention were used as instruments under my direction. They made no independent inventive contribution. The inventive concept, architectural decisions, performance criteria, validation methodology, and reduction to practice were conceived and directed solely by me.

Inventor: Scott Slater **Date:** May 4, 2026

This provisional patent application establishes a priority date for the invention described herein. A non-provisional patent application claiming the benefit of this provisional must be filed within twelve (12) months of the filing date of this provisional pursuant to 35 U.S.C. § 119(e).

Filed under: SES-002-EIDOLON | Docket: Shimmers · Eidolon · SPECTRE System | Priority: April 25, 2026 (parent project SPECTRE)